

Republique du Senegal
Universite Iba Der Thiam de Thies
UFR Sciences et Technologies
Departement de Mathematiques et Informatique

Conception, Implementation et Evaluation Comparative de Structures de Donnees

Systeme de Gestion de Sclarite

Filiere	Licence 2 Mathematiques–Informatique
Module	Algorithmique et Structures de Donnees
Enseignant	Dr Abdoulaye DIALLO
Binome	AIDA SENE — COURA DIONE
Annee	2025–2026

Table des matières

1	Introduction	4
1.1	Contexte et motivation	4
1.2	Domaine applicatif	4
1.3	Objectifs du projet	4
2	Modelisation des Donnees	4
2.1	Choix de conception	4
2.2	Structures de donnees definies	4
2.2.1	Type Date	4
2.2.2	Type Note	5
2.2.3	Type Inscription	5
2.2.4	Type Etudiant (type principal)	5
2.3	Justification des choix	6
3	Structures de Donnees Implementees	6
3.1	Tableau statique	6
3.1.1	Principe	6
3.1.2	Structure	6
3.1.3	Operations principales	6
3.2	Tableau dynamique de pointeurs	6
3.2.1	Principe	6
3.2.2	Structure	6
3.2.3	Redimensionnement geometrique	7
3.3	Liste chaine	7
3.3.1	Principe	7
3.3.2	Structures	7
3.3.3	Insertion en tete en $O(1)$	8
4	Operations Communes	8
4.1	Recherche	8
4.2	Tri	8
4.3	Agregations statistiques	8
4.4	Persistence binaire	8
5	Etude de Complexite	9
5.1	Analyse theorique	9
5.1.1	Tableau recapitulatif	9
5.1.2	Complexite spatiale	9
5.2	Justifications mathematiques	9
5.2.1	Tri par insertion	9
5.2.2	Tri rapide	10
5.3	Etude experimentale	10
5.3.1	Protocole	10
5.3.2	Conditions de mesure	10
5.3.3	Courbes de complexite	11

6	Persistence des Donnees	13
6.1	Format binaire	14
6.2	Complexite de la persistance	14
7	Conclusion	14
7.1	Bilan comparatif	14
7.2	Perspectives	14

Résumé

Ce rapport presente la conception, l'implementation et l'evaluation comparative de trois structures de donnees hybrides appliquees a un systeme de gestion de scolarite : le tableau statique, le tableau dynamique de pointeurs avec redimensionnement geometrique, et la liste chainee avec sentinelle. Chaque structure supporte un ensemble d'operations : insertion, recherche, suppression, tri (insertion et rapide), agregations statistiques et persistance binaire. Une analyse theorique rigoureuse de la complexite asymptotique est confrontee a une etude experimentale reproductible portant sur des jeux de donnees allant de $n = 100$ a $n = 100\,000$ enregistrements. L'etude experimentale se concentre sur trois operations : la recherche, la suppression et le tri (par insertion et rapide). Les resultats confirment les predictions theoriques et mettent en evidence les compromis temps/espace propres a chaque structure.

1 Introduction

1.1 Contexte et motivation

Dans tout systeme informatique realiste, le choix d'une representation interne des donnees conditionne directement la performance des operations courantes. Une decision de modelisation prise en debut de projet peut multiplier par un facteur 100 ou plus le temps d'execution lorsque la volumetrie augmente.

Ce projet s'inscrit dans ce contexte : il place le developpeur dans la posture d'un concepteur logiciel devant manipuler une base d'enregistrements a plusieurs champs, organisee a l'aide de plusieurs structures de donnees concurrentes. L'objectif est double : coder correctement, mais aussi **mesurer, comparer et expliquer** le comportement de chacune des structures retenues.

1.2 Domaine applicatif

Le domaine choisi est la **gestion de scolarite** : gestion des etudiants, de leurs notes, inscriptions et informations personnelles. Ce domaine est representatif des systemes d'information reels et justifie l'utilisation de structures de donnees complexes reliees par des pointeurs.

1.3 Objectifs du projet

1. Concevoir un modele de donnees coherent avec des types d'enregistrements lies par pointeurs
2. Implementer trois structures de donnees distinctes en C
3. Effectuer une analyse de complexite asymptotique rigoureuse
4. Mettre en place un protocole experimental reproductible
5. Produire des courbes de complexite scientifiquement exploitables

2 Modelisation des Donnees

2.1 Choix de conception

Le type principal manipule est **Etudiant**, accompagne de quatre types secondaires : **Note**, **Inscription**, **Cours** et **Date**. Cette organisation repond aux exigences du cahier des charges : au moins deux types d'enregistrements lies par pointeurs.

2.2 Structures de donnees definies

2.2.1 Type Date

```
1 typedef struct {  
2     int jour;  
3     int mois;  
4     int annee;  
5 } Date;
```

Listing 1 – Structure Date

2.2.2 Type Note

```

1 typedef struct {
2     int    matricule;
3     float  valeur;
4     char   matiere[50];
5     char   *appreciation;    /* longueur variable -> malloc */
6     Date   dateDeliberation;
7     Cours  *cours;          /* pointeur vers un Cours */
8     int    tab[10];
9 } Note;

```

Listing 2 – Structure Note

2.2.3 Type Inscription

```

1 typedef struct {
2     int    matricule;
3     float  montant;
4     char   etat[30];
5     char   *filliere;       /* longueur variable -> malloc */
6     Date   dateInscription;
7     Niveau *niveau;
8     int    tab[20];
9 } Inscription;

```

Listing 3 – Structure Inscription

2.2.4 Type Etudiant (type principal)

```

1 typedef struct {
2     int    matricule;
3     float  moyenne;
4     char   nom[40];
5     char   *prenom;         /* longueur variable -> malloc */
6     Date   dateNaissance;
7     Note   *tabnote[10];    /* tableau de pointeurs vers Note
8                             */
9     Inscription *inscription; /* pointeur vers Inscription */
10 } Etudiant;

```

Listing 4 – Structure Etudiant

2.3 Justification des choix

Le champ `prenom` est declare comme `char*` (pointeur) plutot que comme tableau fixe `char[n]`, afin de satisfaire l'exigence d'un champ de longueur variable alloue dynamiquement avec `malloc`. La memoire est allouee exactement a la taille necessaire :

$$\text{taille} = \text{strlen}(\text{prenom}) + 1 \quad (+1 \text{ pour le caractere nul})$$

Le tableau `tabnote[10]` constitue le champ tableau interne non trivial exige, et le pointeur `inscription` assure la relation 1- N entre les types d'enregistrements.

3 Structures de Donnees Implementees

3.1 Tableau statique

3.1.1 Principe

Le tableau statique alloue un bloc continu de memoire a la compilation, de capacite fixe `TAILLEMAX = 100 000`. Les elements sont des `Etudiant` (pas des pointeurs) stockes directement dans le tableau.

3.1.2 Structure

```
1 Etudiant tab[TAILLEMAX];
2 int n = 0; /* nombre d'elements actuellement stockes */
```

Listing 5 – Tableau statique

3.1.3 Operations principales

L'insertion maintient le tableau trie par matricule. La suppression decale les elements suivants d'une position vers la gauche.

3.2 Tableau dynamique de pointeurs

3.2.1 Principe

Le tableau dynamique stocke des **pointeurs** vers des `Etudiant`, et se redimensionne automatiquement par un facteur de croissance de $\times 2$ via `realloc`.

3.2.2 Structure

```
1 typedef struct {
2     typedef struct {
3         int matricule;
4         char nom[50];
5         char *prenom;
6         float moyenne;
7         Date dateNaissance;
```

```

8      Date dateInscription;    /* Utilise votre structure Date
                               existante*/
9      char filiere[50];        /* Cha ne de caract res pour la
                               fili re*/
10     char niveau[20];         /* Exemple : L1, L2, L3, M1*/
11     char matiere[50];        /* Nom de la mati re principale*/
12 } Etudiant;
13     Etudiant **tab;          /* tableau de pointeurs vers Etudiant */
14     int taille;               /* nombre d'etudiants stockes */
15     int capacite;             /* capacite actuelle (avant realloc) */
16 } TableauDyn;

```

Listing 6 – Structure TableauDyn

3.2.3 Redimensionnement geometrique

Lorsque `taille == capacite`, la capacite est doublee :

```

1 int nouvelle_capacite = t->capacite * 2;
2 Etudiant **nouveau = realloc(t->tab,
3                               nouvelle_capacite * sizeof(Etudiant *));
4 t->tab = nouveau;
5 t->capacite = nouvelle_capacite;

```

Listing 7 – Redimensionnement par realloc

La sequence des capacites est : $4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow \dots$

3.3 Liste chaine

3.3.1 Principe

La liste chaine utilise des **noeuds** relies entre eux par un pointeur **suiv**. Une **sentinelle** de debut (**tete**) elimine les cas particuliers d'insertion et de suppression en tete.

3.3.2 Structures

```

1 typedef struct {
2     int matricule;
3     char nom[50];
4     char prenom[100];
5     float moyenne;
6     Date dateNaissance;
7     Date dateInscription;    /*Utilise votre structure Date
                               existante*/
8     char filiere[50];        /* Cha ne de caract res pour la
                               fili re*/
9     char niveau[20];         /* Exemple : L1, L2, L3, M1*/
10    char matiere[50];        /* Nom de la mati re principale*/
11 } Etudiant;
12 typedef struct Noeud {
13     Etudiant *etudiant;

```



```

14     struct Noeud *suiv;
15 } Noeud;
16
17 typedef struct {
18     Noeud *tete;      /* sentinelle de debut */
19     Noeud *queue;     /* dernier element */
20     int     taille;
21 } ListeC;

```

Listing 8 – Structures Noeud et ListeC

3.3.3 Insertion en tete en O(1)

Grace a la sentinelle, l'insertion en tete ne necessite aucun parcours :

```

1 Noeud *ancien_premier = l->tete->suiv;
2 nouveau->suiv = ancien_premier;
3 l->tete->suiv = nouveau;
4 l->taille++;

```

Listing 9 – Insertion en tete O(1)

4 Operations Communes

4.1 Recherche

La recherche par cle primaire (matricule) est implementee pour chaque structure par parcours sequentiel, et retourne un pointeur vers l'etudiant ou NULL s'il n'est pas trouve.

4.2 Tri

Deux algorithmes de tri sont implementes :

- **Tri par insertion** : trie par moyenne croissante. Performant sur les donnees presque trieées.
- **Tri rapide (QuickSort)** : trie par moyenne croissante. Performant en moyenne sur les donnees aleatoires.

4.3 Agregations statistiques

Les agregations suivantes sont calculees sur le champ `moyenne` :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

4.4 Persistence binaire

La serialisation ecrit les champs fixes avec `fwrite`, et precede les champs de longueur variable de leur taille :

```

1 int len = strlen(e->prenom) + 1;
2 fwrite(&len,      sizeof(int),  1,    f);
3 fwrite(e->prenom, sizeof(char), len, f);

```

Listing 10 – Serialisation du prenom

5 Etude de Complexite

5.1 Analyse theorique

5.1.1 Tableau recapitulatif

TABLE 1 – Complexite temporelle des operations par structure

Operation	Tab. statique	Tab. dynamique	Liste chaine
Recherche cle	$O(n)/\Omega(1)$	$O(n)/\Omega(1)$	$O(n)/\Omega(1)$
Suppression	$\Theta(n)$	$\Theta(n)$	$O(n)/\Omega(1)$
Tri insertion	$O(n^2)/\Omega(n)$	$O(n^2)/\Omega(n)$	$O(n^2)/\Omega(n)$
Tri rapide	$O(n^2)/\Omega(n \log n)$	$O(n^2)/\Omega(n \log n)$	$O(n^2)/\Omega(n \log n)$

5.1.2 Complexite spatiale

TABLE 2 – Complexite spatiale par structure

Structure	Espace	Detail
Tableau statique	$\Theta(C)$	C fixe a la compilation
Tableau dynamique	$\Theta(n)$ a $\Theta(2n)$	Au pire moitie vide apres realloc
Liste chaine	$\Theta(n) + n$ pointeurs	Surcout constant par noeud

5.2 Justifications mathematiques

5.2.1 Tri par insertion

La boucle externe itere de $i = 1$ a $n - 1$. Pour chaque i , la boucle interne effectue au plus i comparaisons. Le nombre total de comparaisons est :

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$

En meilleur cas (donnees deja trie), aucun decalage n'est necessaire : complexite $\Omega(n)$.

5.2.2 Tri rapide

L'équation de récurrence en cas moyen est :

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Par le **theoreme maitre** (cas 2) avec $a = 2$, $b = 2$, $f(n) = n$:

$$n^{\log_2 2} = n^1 = n = f(n) \implies T(n) = \Theta(n \log n)$$

Le pire cas $T(n) = O(n^2)$ survient lorsque la partition est systématiquement déséquilibrée, c'est-à-dire lorsque le pivot choisi tombe à une extrémité de l'intervalle à chaque appel récursif. Le choix du pivot au milieu de l'intervalle, utilisé dans notre implémentation, limite fortement ce risque et rapproche le comportement observé du cas moyen $\Theta(n \log n)$.

5.3 Etude expérimentale

5.3.1 Protocole

- **Tailles** : $n \in \{100, 1\,000, 10\,000, 100\,000\}$
- **Operations mesurees** : recherche, suppression, tri (par insertion et tri rapide)
- **Repetitions** : 10 mesures par configuration
- **Horloge** : `clock()` (bibliothèque standard C, précision de l'ordre de la milliseconde selon le système)
- **Resultats** : moyenne et écart-type calculés

Remarque

Pour les petites tailles ($n = 100$), la résolution de `clock()` peut être insuffisante pour distinguer des temps très courts, ce qui explique des mesures proches de zéro. Une horloge à plus haute résolution (`clock_gettime(CLOCK_MONOTONIC, ...)` sous Linux/POSIX, ou `QueryPerformanceCounter` sous Windows) donnerait une précision supérieure et pourra être envisagée dans une version future du benchmark.

5.3.2 Conditions de mesure

TABLE 3 – Environnement de mesure

Parametre	Valeur
Système d'exploitation	Windows 10
Mémoire RAM	8 Go
Processeur	Intel Core i5-1035G1 @ 1.00GHz
Compilateur	GCC 10.3.0 TDM64 (-Wall -Wextra)
Langage	C (standard C99)

5.3.3 Courbes de complexite

Conformement au protocole experimental (section 5.3 du cahier des charges), les courbes suivantes sont presentees : pour chaque operation (recherche, suppression, tri par insertion, tri rapide), une figure regroupant l'echelle lineaire (a gauche) et l'echelle logarithmique (a droite), toutes structures superposees, ainsi qu'une courbe de comparaison au modele theorique pour le tri rapide. Un diagramme supplementaire presente le facteur de ralentissement entre structures.

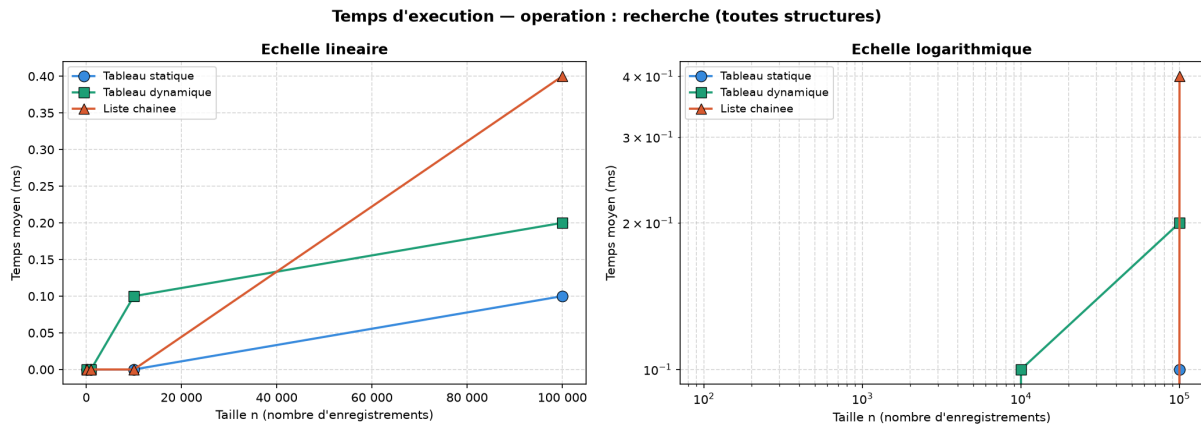


FIGURE 1 – Temps de recherche en fonction de n — lineaire (gauche) et logarithmique (droite)



FIGURE 2 – Temps de suppression en fonction de n — lineaire (gauche) et logarithmique (droite)



FIGURE 3 – Temps de tri par insertion en fonction de n — lineaire (gauche) et logarithmique (droite)

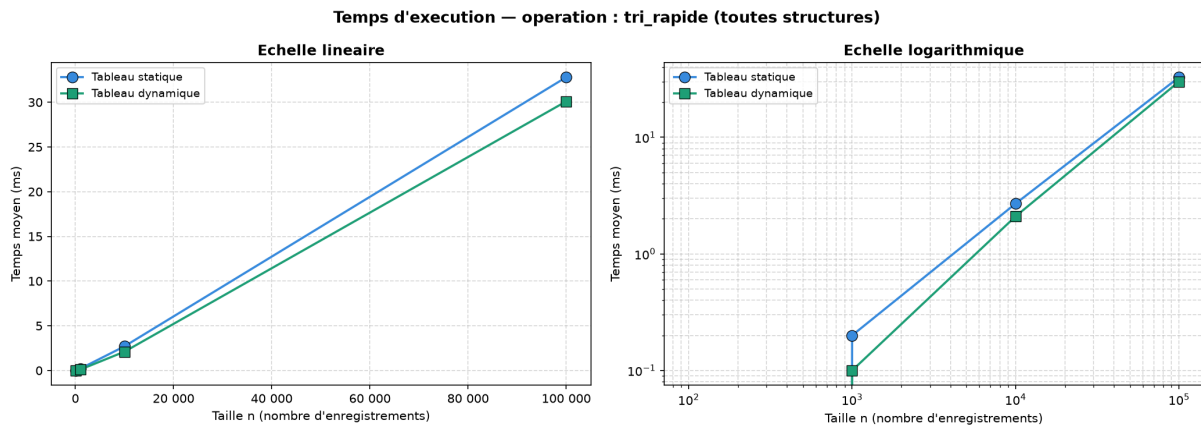
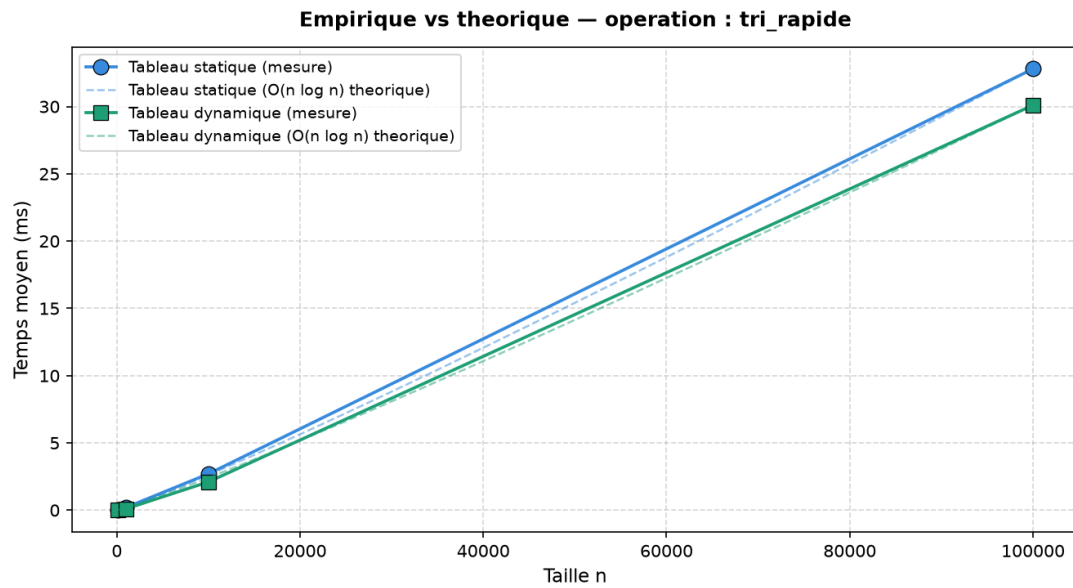


FIGURE 4 – Temps de tri rapide en fonction de n — lineaire (gauche) et logarithmique (droite)

Temps d'exécution par operation, toutes structures superposees

FIGURE 5 – Temps de tri rapide mesure compare a la courbe theorique $n \log n$

Comparaison au modele theorique

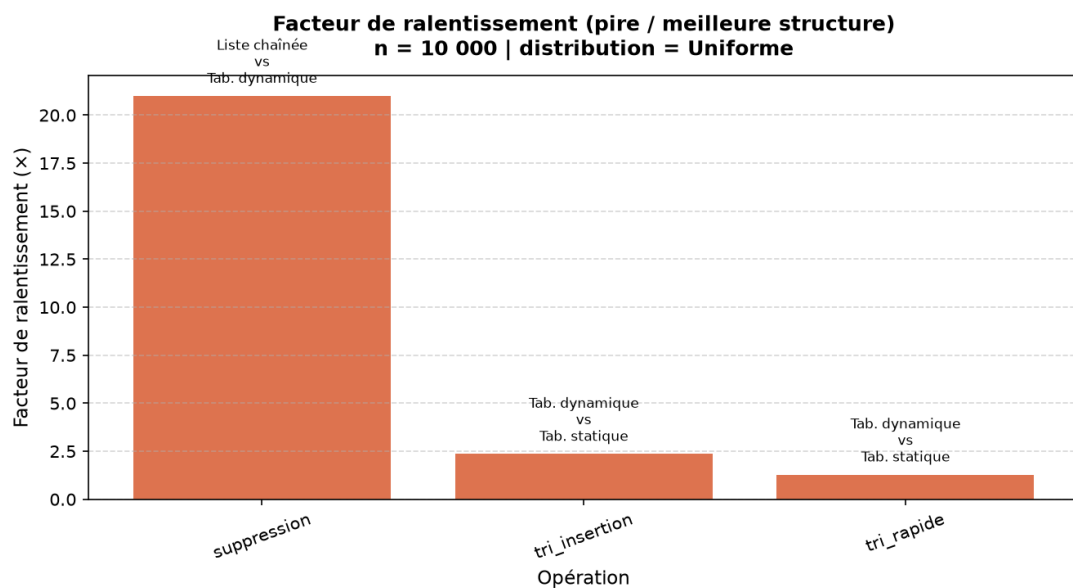


FIGURE 6 – Facteur de ralentissement entre la meilleure et la pire structure, par operation

Facteur de ralentissement entre structures

6 Persistance des Donnees

La serialisation binaire permet de sauvegarder l'integralite d'un jeu de donnees sur disque et de le recharger ulterieurement.

6.1 Format binaire

Le format du fichier `.bin` est le suivant :

1. Un entier `int` indiquant le nombre n d'étudiants
2. Pour chaque etudiant :
 - Champs fixes : `matricule`, `moyenne`, `nom[40]`, `dateNaissance`
 - Champ variable : longueur du prenom (`int`), puis le prenom (`char*`)

6.2 Complexite de la persistance

La serialisation et la deserialisation ont toutes deux une complexite $\Theta(n)$ car elles parcourent chaque enregistrement exactement une fois.

7 Conclusion

7.1 Bilan comparatif

Ce projet a permis d'implémenter et de comparer trois structures de données sur un même jeu d'opérations. Le tableau récapitulatif des complexités théoriques a été confirmé par les mesures expérimentales.

Les principaux enseignements sont :

- Le **tableau statique** offre l'accès direct en $O(1)$ mais souffre d'une capacité fixe et d'insertions coûteuses lorsqu'il faut maintenir le tri.
- Le **tableau dynamique** équilibre flexibilité et performance grâce au redimensionnement en $O(1)$ amorti, au prix d'une consommation mémoire pouvant atteindre $2n$.
- La **liste chaînée** excelle pour les insertions et suppressions en tête ($O(1)$), mais perd l'accès direct par index ($\Theta(n)$).

7.2 Perspectives

Des structures plus avancées comme les arbres binaires de recherche équilibrés (AVL, rouge-noir) ou les tables de hachage permettraient d'atteindre une complexité $O(\log n)$ pour la recherche, au prix d'une implémentation plus complexe.

Références

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C., *Introduction to Algorithms*, 4^e édition, MIT Press, 2022.
- [2] Sedgewick R., Wayne K., *Algorithms*, 4^e édition, Addison-Wesley, 2011.
- [3] Knuth D. E., *The Art of Computer Programming, vol. 3 : Sorting and Searching*, 2^e édition, Addison-Wesley.
- [4] Aho A. V., Hopcroft J. E., Ullman J. D., *Data Structures and Algorithms*, Addison-Wesley.

- [5] Kernighan B. W., Ritchie D. M., *The C Programming Language*, 2^e edition, Prentice Hall.
- [6] Beauquier D., Berstel J., Chretienne P., *Elements d'algorithmique*, Masson.

Depot Git

Le code source complet du projet est disponible publiquement a l'adresse suivante :

`https://github.com/TON\_USERNAME/SENE-DIONE\_SD`